

Motivation

Calls to function templates are occasionally unable to exploit ADL, because when specifying a *template-id* in such cases without any suitably declared template in reach, the < token is treated as the less-than operator. Consider the example in core issue 687^[CWG687]:

```
namespace N {
    struct A { };
    template <typename T>
    T func(const A&) { return T(); }
}

void f() {
    N::A a;
    func<int>(a);    // error
}
```

Qualifying func is possible but has different semantics. Instead, it would be optimal to apply the template keyword in order to disambiguate. This paper proposes to permit the template keyword in function calls by augmenting the definition of *postfix-expressions* accordingly.

Note that the *literal-operator-id* case is included purely for consistency, as all involved types are fundamental (cf. [over.literal] ¶5) (i.e. ADL is inapplicable).

Wording

Augment 5.2 [expr.post] ¶1 as indicated:

postfix-expression:

```
template template-id ( expression-listopt )
```

Modify 14.2 [temp.names] ¶2 as indicated:

For a *template-name* to be explicitly qualified by the template arguments, the name must be known to refer to a template **or be prefixed by the template keyword**.

Modify 14.2 [temp.names] ¶3 as indicated:

After name lookup (3.4) finds that a name is a *template-name* or that an *operator-function-id* or a *literal-operator-id* refers to a set of overloaded functions any member of which is a function template, **or the name is prefixed by the keyword template**, if **this the name** is followed by a <, the < is always taken as the delimiter of a *template-argument-list* and never as the less-than operator.

Augment 14.2 [temp.names] ¶4 as indicated:

[...] [*Example*:

```
struct X {
    template <std::size_t> X* alloc();
    template <std::size_t> static X* adjust();
};
template <class T> void f(T* p) {
    T* p1 = p->alloc<200>();           // ill-formed: < means less than
    T* p2 = p->template alloc<200>();  // OK: < starts template argument list
    T::adjust<100>();                  // ill-formed: < means less than
    T::template adjust<100>();         // OK: < starts template argument list
}

template <typename T>
struct B {
    template <typename U>
    void f();
};
```

```

template <typename T>
struct D : B<T> {
    void g() {
        f<T>(); // ill-formed: < means less than
        template f<T>(); // OK: < starts template argument list
    }
};

template <typename T>
struct E {
    void bar() {
        h<int>(T{}); // ill-formed: < means less than
        template h<int>(T{}); // OK: < starts template argument list.
                                // h can be found by argument-dependent name lookup
                                // in the template instantiation context.
    }
};

```

— end example]

Modify 14.2 [temp.names] ¶5 as indicated (this simultaneously resolves core issue 1794^[CWG1794]):

A name prefixed by the keyword `template` shall be a *template-id* or the name shall refer to a class template **or alias template**. [*Note*: The keyword `template` may not be applied to non-template members of class templates. — end note] [*Note*: As is the case with the `typename` prefix, the `template` prefix is allowed in cases where it is not strictly necessary; i.e., when the *nested-name-specifier* or the expression on the left of the `->` or `.` is not dependent on a *template-parameter*, or **the use does not appear in the scope of a template** **the subsequent name would be found to refer to a template via unqualified name lookup (3.4.1)**. — end note]

NB: The first note refers to `tempoids` and should stay as is.

Acknowledgments

The author would like to thank Daveed Vandevoorde for assistance in preparing the wording.

References

[CWG1794] “`template` keyword and alias templates”: wg21.link/cwg1794

[CWG687] “`template` keyword and alias templates”: wg21.link/cwg687

[EWG92] “[tiny] Core issue 687, `template` keyword with *unqualified-ids*”: wg21.link/ewg92